



ТЗ

ТЗ-Ревьюер

AI-инструмент для предварительного ревью технических заданий
на разработку новых потоков и витрин данных

Кейс МТС · продукт NET

Команда 6 · финальная защита

Проверено на реальных документах кейсодателя, вживую с LLM

AI Engineer: Кайаал Яхья · Караташоглу Фырат

Часть проблем в ТЗ находится уже на разработке

Объём ручной вычитки

Аналитики и разработчики вручную вычитывают объёмные ТЗ перед постановкой задачи.

Неоднозначность и пробелы

Расплывчатые формулировки и недостающие требования не всегда видны при чтении.

Нет единого ревью

Проверка зависит от опыта и загрузки конкретного человека.

Когда проблема всплывает поздно — запускается цикл повторной работы:

Аналитик правит ТЗ



Разработчик переделывает реализацию

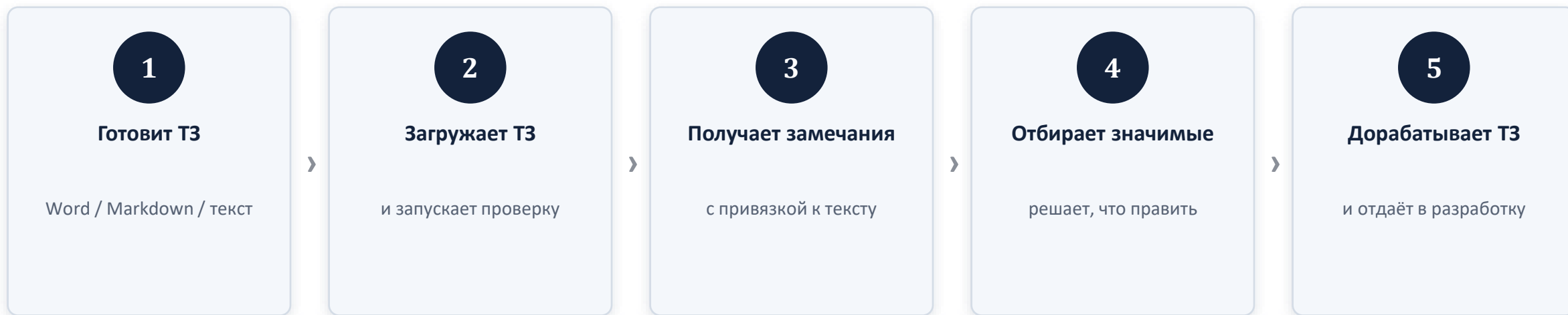


Тестировщик перепроверяет сценарии

Цена проблемы = число поздних уточнений на одно ТЗ × трудоёмкость цикла уточнения × число ТЗ в квартал

Второе мнение по ТЗ до передачи в разработку

Основной пользователь — аналитик NET. Дополнительное ревью, не замена аналитика и не вердикт о готовности.



Проверено не только на синтетике, но и на реальных ТЗ кейсодателя

- весь сценарий прогнан на 3 реальных документах МТС в офлайн- и LLM-режиме;
- вторичные пользователи: разработчик и тимлид — быстрый входной чек перед оценкой задачи.

Что должно делать решение

- находить конкретные фрагменты, непонятные разработчику;
- определять недостающую и неоднозначную информацию;
- объяснять, почему фрагмент требует внимания;
- предлагать, что уточнить или дополнить;
- показывать, к какому разделу ТЗ относится замечание;
- формировать общий результат проверки документа.

КЛЮЧЕВОЙ ПРИНЦИП

Замечание привязано к конкретному фрагменту ТЗ, а не является общим советом.

Подтверждено на практике: на реальных документах кейсодателя абсолютное большинство замечаний LLM-режима содержали дословную цитату.

Пайплайн предварительного ревью

1

Разбор документа

.md / .txt / .docx с таблицами → разбивка на разделы

2

Покрытие шаблона

официальный шаблон МТС, 21 раздел — что не найдено в ТЗ

3

LLM-ревью по рубрике

23 категории, включая 4 по официальным критериям МТС

4

Эвристики (без LLM)

заглушки, «и т.д.», поле без типа/NOT NULL, Kafka без кластера

5

Структурированный отчёт

замечания, критичность, вопросы аналитику, покрытие шаблона

ОФИЦИАЛЬНЫЕ КРИТЕРИИ МТС

- спецификация сериализации входа/выхода;
- прямая ссылка на Data Catalog;
- NOT NULL / NULLABLE по каждому полю;
- Kafka-кластер и полный путь в HDFS;
- перечень используемых справочников (НСИ).

ПРОВАЙДЕР-НЕЗАВИСИМОСТЬ

Anthropic · любой OpenAI-совместимый (OpenAI, Gemini, OpenRouter, локальные модели) · офлайн-режим.
Проверено вживую на 2 провайдерах.

3 документа кейсодателя, 2 режима анализа

Поток геолокации

модуль реального времени, 2 источника, 5 целевых таблиц

Поток CDR

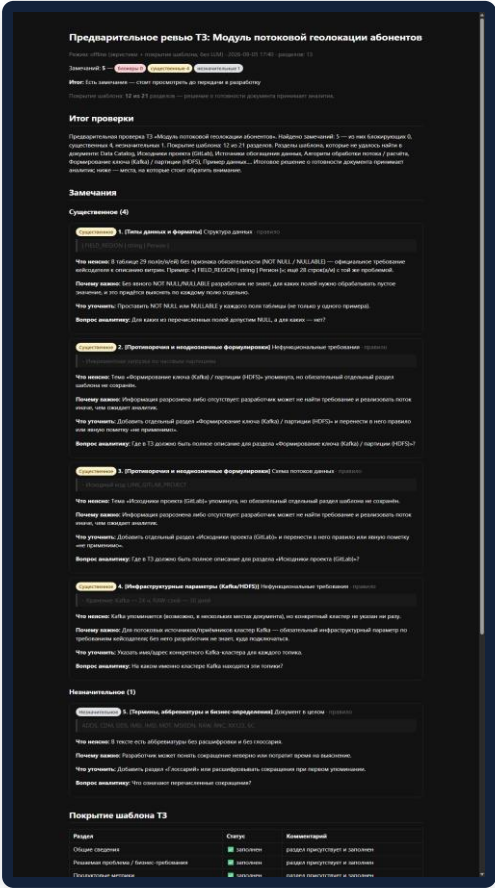
prepaid-звонки/SMS, фильтрация и коррекция часового пояса

Витрина-агрегат

ежемесячная сводка по устройствам — образец финального теста

2 РЕЖИМА, ОБА БЕЗ СБОЕВ

Офлайн (эвристики + покрытие шаблона) и живая LLM — Google Gemini 2.5 Flash через OpenAI-совместимый эндпоинт, без единой правки кода.



Реальный вывод: офлайн-режим, поток геолокации (настоящий документ кейсодателя).

Реальное противоречие в документе кейсодателя

«Модуль потоковой геолокации абонентов»

Раздел «Продуктовые метрики»: «Задержка: < 1 мин»

Раздел «Нефункциональные требования»: «Стриминг в реальном времени (задержка ≈ 0 сек)»

КАТЕГОРИЯ

Противоречия и неоднозначные формулировки

КРИТИЧНОСТЬ

blocker

ЧТО НЕЯСНО

Два раздела одного документа дают разные требования к задержке потока.

ПОЧЕМУ ВАЖНО

Неоднозначность в целевом SLA может привести к неверному выбору архитектурных решений.

ВОПРОС АНАЛИТИКУ

Какое точное требование к задержке потока данных?

Найдено автоматически, LLM-режим (Google Gemini). Найти такое при беглом чтении объёмного ТЗ — легко упустить; на разработке — уже поздно.

Реальные данные вскрыли реальные баги — все исправлены

Дублирующиеся находки

Найдено: LLM и офлайн-правило независимо находили один и тот же пробел (Kafka/HDFS/Data Catalog) — попадали в отчёт дважды.

Исправлено: Дедупликация по теме находки, а не только по точной цитате.

Нераспознанные типы данных

Найдено: Поля типа string/long/tinyint ложно помечались как «тип не указан».

Исправлено: Список распознаваемых типов расширен по реальным данным.

Спам одинаковой находкой

Найдено: «Нет NOT NULL» повторялось до 20 раз — по одной находке на строку таблицы.

Исправлено: Одна сводная находка на таблицу вместо N одинаковых.

Ложные срабатывания

Найдено: Заголовок таблицы принимался за «поле без типа»; КАПС-слово — за аббревиатуру.

Исправлено: Распознавание заголовков таблиц; аббревиатура — только при повторе в тексте.

Тестирование на реальных документах — не формальность: все 4 проблемы отсутствовали на синтетических примерах и проявились только на реальных данных кейсодателя.

Что подтвердилось, что осталось риском

ПОДТВЕРДИЛОСЬ НА ТЕСТАХ

- замечания привязаны к цитате — на всех 3 реальных ТЗ;
- провайдер-независимость — переключение на Gemini без правок кода;
- находит содержательные противоречия, не только механику;
- офлайн-режим даёт полезный результат без LLM.

ПРОВЕРЕНО КАЧЕСТВЕННО, НЕ В ЦИФРАХ

- качество замечаний оценено вручную (эксперт прочитал все отчёты) — не переведено в precision/recall, для этого нужна разметка эталонных находок;
- нет пост-проверки, что цитата дословно есть в тексте (пока — визуальная сверка);
- few-shot построен на обобщённом опыте, не на реальных правках NET — их ещё предстоит получить.

КЛЮЧЕВЫЕ РИСКИ AI-РЕШЕНИЯ

Ложные/дублирующиеся находки

→ критичность и сортировка; дедупликация по теме (проверено и исправлено на реальных данных)

AI выносит вердикт о готовности

→ числовая оценка убрана из вывода; отчёт показывает только факты

Зависимость от одного провайдера

→ провайдер-независимый клиент; офлайн-режим как страховка

Закрытый контур / чувствительные данные

→ локальные эндпоинты и офлайн-режим поддержаны архитектурно

ОГРАНИЧЕНИЯ И ПЛАНЫ

Что дальше — по пути к пилоту

ОГРАНИЧЕНИЯ

- ТЗ подаётся как связный текст (Word/Markdown), не скан;
- покрытие шаблона — по ключевым словам заголовков, возможны ложные срабатывания на нестандартных формулировках;
- нет обученного эталонного датасета для точных числовых метрик качества.

СЛЕДУЮЩИЙ ШАГ К ПИЛОТУ

- 1 Собрать реальные корректировки NET → обновить базу паттернов
- 2 Пост-проверка цитат + подсветка фрагмента в тексте
- 3 Измерить прокси-метрики на размеченной выборке
- 4 API-сервис вокруг ядра для встраивания в процесс
- 5 Слепой пилот с 2–3 аналитиками NET на 1–2 спринтах

ВЫВОД ХАКАТОНА

Гипотеза подтвердилась на реальных данных: инструмент находит содержательные, а не только механические проблемы, и делает это одинаково надёжно на разных LLM-провайдерах.

Команда 6 — распределение задач

Кайаал Яхья

AI Engineer · участие 50%

- рубрика ревью (23 категории) и промпты, включая интеграцию официальных критериев МТС
- структурированный вывод и сборка отчёта
- тестирование на 3 реальных документах в офлайн- и LLM-режиме, диагностика и исправление найденных багов
- проектная документация и презентация

Караташоглу Фырат

AI Engineer · участие 50%

- архитектура решения, сборка MVP, провайдер-независимый клиент LLM
- разбор документа: иерархия разделов, таблицы и ссылки из .docx
- проверка покрытия шаблона: 5 статусов раздела (заполнен / пустой / не применимо / упомянут / нет), структурные пробелы как отдельные замечания
- устойчивость демо (сохранение результата в Streamlit) и расширенный набор автотестов
- независимый прогон проекта с нуля на отдельной машине + критическое ревью качества LLM-выдачи

Архитектурные решения и оценка качества — совместно, компоненты интегрированы в общий пайплайн.